

SHELL Programmierung

Susanne Schaller, MMSc - susanne.schaller@fh-hagenberg.at

Andreas Scheibenpflug, MSc - andreas.scheibenpflug@fh-hagenberg.at



Shell Stellungsparameter

Einem Shell-Skript können beim Aufruf wie jedem anderen Befehl Argumente übergeben werden.

Die Shell stellt für das Arbeiten mit den Argumenten die Stellungsparameter sowie einige spezielle Parameter zur Verfügung.

\$1, \$2, \$3, ... sind die Stellungsparameter. In \$1 ist das erste Argument, in \$2 das zweite Argument und so weiter. Man kann ihnen keinen Wert direkt zuordnen.

Von den speziellen Parametern sind **\$***, **\$@** und **\$#** für das Arbeiten mit den Stellungsparametern wichtig.

Shell Spezielle Parameter

Den sogenannten speziellen Parametern kann kein Wert zugewiesen werden und sie werden von Der Shell speziell behandelt:

\$*

Expandiert zu den Stellungsparametern. Werden doppelte Anführungszeichen benutzt ("\$*"), so wird zu einem einzigen Wort expandiert. Die Werte der Stellungsparameter sind durch das erste Zeichen von IFS (normalerweise das Leerzeichen) getrennt (also "\$1 \$2 \$3 ...").

\$@

Expandiert zu den Stellungsparametern. Werden doppelte Anführungszeichen benutzt ("\$@"), so expandiert jeder Parameter zu einem eigenen Wort (also "\$1" "\$2" "\$3" ...).

\$#

Expandiert zur Anzahl der Stellungsparameter.

\$?

Expandiert zum Rückgabewert des letzten, im Vordergrund ausgeführten Befehls.

\$\$

Expandiert zur PID der Shell (auch wenn in einer Subshell verwendet).

\$0

Expandiert zum Namen der Shell oder des Shell-Skripts.

Beispiele

```
#!/bin/sh
# This is a comment!
echo "I was called with $# parameters"
echo "My name is $0" # This is also a comment!
echo "My first parameter is $1"
echo "My second parameter is $2"
echo "All parameters are $@ "
echo "My internal process ID is $$"
echo "Exit code of last command $?"
```

```
osboxes@osboxes:~/Desktop/stuff$ ./script22.sh Test Susi Hello
I was called with 3 parameters
My name is ./script22.sh
My first parameter is Test
My second parameter is Susi
All parameters are Test Susi Hello
My internal process ID is 3362
Exit code of last command 0
```

Shell Skript: Aufbau

In einem Shell-Skript werden meist überwiegend Shell-Befehle benutzt. Zu den Shell-Befehlen gehören unter anderem:

- Variablenzuweisungen
- Tests (`test`, `[ ... ]`, `[[ ... ]]`)
- Verzweigungen (`if`, `case`)
- Schleifen (`for`, `while`, `until`)
- Ein- und Ausgabebefehle (`read`, `echo`, `printf`)
- Funktionen (`function`)

Darüber hinaus können natürlich auch alle anderen Programme wie `head`, `tail`, `cut`, ... verwendet werden.

Anfang eines Skriptprogramms steht die **Shebang** Line oder auch **Magic Line**

Shebang Line

```
#!/bin/sh
```

```
#!/bin/bash
```

```
#!/usr/bin/perl -w
```

```
use strict;  
use Gtk2 '-init';
```

Diese Zeile weist das Betriebssystem an, diese Datei mit dem Interpreter-Programm /bin/sh, in diesem Fall also der Standard-Unix-Shell, auszuführen.

Interaktive Eingabe

- Interaktive Eingabe von Daten in SHELL-Skript
 - `read variable [variable ...]`
 - Liest Werte von *stdin* und weist Werte den angegebenen Variablen zu
 - Getrennt durch *IFS* Zeichen
 - Falls mehr Variable als Werte → Rest mit Leerstrings belegt
 - Falls mehr Werte als Variable → Letzte Variable bekommt Rest der Werte als Zeichenkette zugewiesen
- Beispiel:

```
#!/bin/sh
echo "What's your name? "
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

If Anweisung

- `test Bedingung` oder `[ Bedingung ]`
 - Prüft Bedingung, liefert 0 falls true, 1 falls false
- `if - then - else`
 - `if bedingung`
 `then`
 `kommandos`
 `fi`
 - `if bedingung`
 `then`
 `kommandos`
 `else`
 `kommandos`
 `fi`
 - `if bedingung1`
 `then`
 `kommandos`
 `elif bedingung2`
 `then`
 `kommandos`
 `elif .....`
 `fi`

Bedingungen testen

- Zwei Möglichkeiten
 - Verwendung des Kommandos test
 - test Bedingung
 - Bedingung in eckige Klammern einfügen. Dabei muss **vor** und **nach** jeder Klammer ein Leerzeichen stehen.
[Bedingung]
- In beiden Fällen wird keine Ausgabe produziert, sondern nur ein Rückgabewert geliefert
 - Rückgabewert 0: Bedingung ist wahr
 - Rückgabewert 1: Bedingung ist falsch

Optionen zum Testen von Zahlenwerten

- Variablen können Integerwerte enthalten
- Integer können mit Hilfe von folgenden Operatoren verglichen werden

-eq	equals: Liefert ‚true‘ falls beide Zahlen den gleichen Wert haben.
-ne	not equals: Liefert ‚true‘ falls beide Zahlen einen unterschiedlichen Wert haben.
-lt	less than: Liefert ‚true‘ falls <i>zahl1</i> kleiner als <i>zahl2</i> ist.
-le	less or equal than: Liefert ‚true‘ falls <i>zahl1</i> kleiner oder gleich <i>zahl2</i> ist.
-gt	greater than: Liefert ‚true‘ falls <i>zahl1</i> größer als <i>zahl2</i> ist.
-ge	greater or equal than: Liefert ‚true‘ falls <i>zahl1</i> größer oder gleich <i>zahl2</i> ist.

Beispiele Anweisungen

- Testen, ob die Anzahl der an des Shellscript übergebenen Argumente gleich 0 ist. (\$# - Parameter)

```
#!/bin/sh
if [ $# -eq 0 ]
then
    echo "No arguments defined"
else
    echo "Number of arguments is: $#"
```

```
#!/bin/sh
if test $# -eq 0
then
    echo "No arguments defined"
else
    echo "Number of arguments is: $#"
```

```
#!/bin/sh
test $# -eq 0
if [ $? -eq 0 ] ; then
    echo "No arguments defined"
else
    echo "Number of arguments is: $#"
```

```
#!/bin/sh
if [ $# -eq 0 ]; then
    echo "No arguments defined"
else
    echo "Number of arguments is: $#"
```

Testoptionen auf Dateien

```
osboxes@osboxes: ~/Desktop/stuff
TEST(1)                                User Commands

NAME
    test - check file types and compare values

SYNOPSIS
    test EXPRESSION
    test
        [ EXPRESSION ]
        [ ]
        [ OPTION ]

DESCRIPTION
    Exit with the status determined by EXPRESSION.
```

- Nur für Dateien/Verzeichnisse verwendbar
- Existiert Datei/VZ nicht, wird automatisch false zurückgeliefert.

Syntax:: „test *option dateiname*“

-d	Prüft, ob es sich bei der Datei um ein Verzeichnis handelt
-e	Prüft, ob die angegebene Datei im Dateisystem existiert
-f	Prüft, ob die angegebene Datei eine reguläre (normale) Datei ist
-r	Prüft, ob der aktuelle Benutzer Leserechte auf der Datei besitzt
-w	Prüft, ob der aktuelle Benutzer Schreibrechte auf der Datei besitzt
-x	Prüft, ob der aktuelle Benutzer Exekuterechte auf der Datei besitzt
-s	Prüft, ob die Datei nicht leer ist
	u.v.a.m

Testoptionen für Dateivergleiche & Zeichenketten

Syntax: „test *dateiname1* option *dateiname2*“

-nt	newer than: Prüft, ob <i>dateiname1</i> neuer ist als <i>dateiname2</i> . Für den Vergleich wird das letzte Änderungsdatum hergenommen, nicht das Erstellungsdatum!
-ot	older than: Prüft, ob <i>dateiname1</i> älter ist als <i>dateiname2</i> . Für den Vergleich wird das letzte Änderungsdatum hergenommen, nicht das Erstellungsdatum!
-ef	Prüft, ob die beiden Dateien die selbe Inode-Nummer haben (Hardlinks).

- Diese Optionen erlauben den Vergleich zweier Zeichenketten.

Syntax: „test *zeichenkette1* option *zeichenkette2*“

=	Liefert ‚true‘ wenn die beiden Zeichenketten gleich sind.
!=	Liefert ‚true‘ wenn die beiden Zeichenketten unterschiedlich sind.

Syntax: „test option *zeichenkette*“

-z	Liefert ‚true‘ wenn die Zeichenkette null Zeichen lang (leer) ist.
-n	Liefert ‚true‘ wenn die Zeichenkette zumindest ein Zeichen enthält (nicht leer ist).

Logisches Verknüpfen von mehreren Bedingungen

- Mit Hilfe der folgenden Optionen lassen sich einzelne Bedingungen logisch miteinander verbinden.
- Können mit runden Klammern () auch mehrfach verbunden werden, runde Klammern müssen durch Voranstellen eines Backslashes \ vor der Interpretation durch die Shell geschützt werden
- Normale Bedeutung runder Klammern: geklammerte Befehle werden in eigener Shell ausgeführt

<i>! bedingung</i>	Negieren der Bedingung: Liefert ‚true‘ wenn die Bedingung selbst ‚false‘ liefert und umgekehrt.
<i>bedingung1 -a bedingung2</i>	Logisches UND: Liefert ‚true‘ wenn beide Bedingungen ‚true‘ ergeben. Falls eine der beiden Bedingungen ‚false‘ ergibt, liefert das logische UND auch ‚false‘.
<i>bedingung1 -o bedingung2</i>	Logisches ODER: Liefert ‚true‘ wenn eine der beiden Bedingungen ‚true‘ ergibt oder wenn beide ‚true‘ ergeben. ‚false‘ wird nur dann geliefert, falls beide Bedingungen ‚false‘ ergeben.

For-Anweisung

- Mit Hilfe der for-Schleife kann man eine Anweisungsfolge in der SHELL für unterschiedliche Belegungen einer Variablen ausführen.
 - `for selector in liste`
do
 kommandofolge
done
 - `for selector`
do
 kommandofolge
done

For Anweisung ohne Liste

- Wird keine Werteliste angegeben, so nimmt die SHELL als Werteliste die übergebenen Parameter.
- Jeder Parameter wird als Zeichenkette interpretiert und der Schleifen-Variable übergeben.
- Kein Parameter -> Schleife wird nicht durchlaufen.
- Beispiel:

```
#!/bin/sh
for PARAM
do
    echo $PARAM
done
```


Zeilenweises Lesen einer Datei

```
#!/bin/sh
for line in `cat file.txt`
do
    echo $line
done
```

Ausgabe:

Das
ist
Zeile
1
Das
ist
Zeile
2

```
#!/bin/sh
while read line
do
    echo $line
done < file.txt
```

Ausgabe:

Das ist Zeile 1
Das ist Zeile 2

while- und until-Anweisung

- (1) while *Bedingung*
 do
 Kommandofolge
 done
- (2) until *Bedingung*
 do
 Kommandofolge
 done

Bedeutung

- (1) solange Bedingung erfüllt ist, mache Kommandoliste
- (2) bis Bedingung erfüllt ist, mache Kommandoliste

Beispiele

```
#!/bin/sh
X=""
while [ "$X" != "end" ]
do
    echo -n " Eingabe:  "
    read X
    echo "Ausgabe: $X"
done
```

```
osboxes@osboxes:~/Desktop/stuff$ ./script3.sh
Eingabe: Hello
Ausgabe: Hello
Eingabe: Testing
Ausgabe: Testing
Eingabe: Loops
Ausgabe: Loops
Eingabe: end
Ausgabe: end
```

```
#!/bin/sh
# Schleife bricht ab, wenn N > 10
N=1
while [ $N -le 10 ]
do
    echo "N hat jetzt den Wert $N"
    N=`expr $N + 1`
done
```

```
osboxes@osboxes:~/Desktop/stuff$ ./script3.sh
N hat jetzt den Wert 1
N hat jetzt den Wert 2
N hat jetzt den Wert 3
N hat jetzt den Wert 4
N hat jetzt den Wert 5
N hat jetzt den Wert 6
N hat jetzt den Wert 7
N hat jetzt den Wert 8
N hat jetzt den Wert 9
N hat jetzt den Wert 10
```

Ergebnisabhängige Befehlsausführung

- Wenn ein Befehl ausgeführt wurde gibt er standardmäßig einen Fehlercode zurück
 - Bei erfolgreicher Ausführung ist dies der Wert 0
 - Bei einem Fehler der Wert > 0
 - Abfrage des Fehlercodes mit **echo \$?** möglich
- Fehlercode wird verwendet um Befehle abhängig von dessen Ergebnis miteinander zu verknüpfen
 - `command1 && command2`
 - Bei dieser Verknüpfung wird `command2` nur ausgeführt wenn der `command1` erfolgreich ausgeführt wurde, also Fehlercode 0 geliefert hat. (Befehle sind mit einem logischen UND (AND) verknüpft.)
 - `command1 || command2`
 - Bei dieser Verknüpfung wird der `command2` nur ausgeführt wenn der `command1` einen Fehler verursacht hat, also den Fehlercode >0 geliefert hat. (Befehle mit einem logischen ODER (OR) verknüpft.)

Beispiele

```
DIR=.  
test -d $DIR  && echo $DIR  is a dir || echo $DIR is no dir
```

```
X=/bin/ls  
[ -x "$X" ] && echo "$X is the path of an executable file"
```

```
touch newfile  
X=newfile  
[ $X -nt "/etc/passwd" ] && echo "X is a file which is \  
newer than /etc/passwd"
```

```
mkdir mydir  
X=mydir  
[ -f $X ] && \  
    echo "X is the path of a real file" || \  
    echo "No such file: $X"
```

Beispiele

```
X=hello
[ $X = "hello" ] && \
    echo "X matches the string \"hello\""
```

```
X=World
[ $X != "hello" ] && \
    echo "X does not match the string \"hello\""
```

```
X=World
[ -n $X ] && echo "X is of nonzero length"
```

```
X=""
[ -z $X ] && echo "X is of zero length"
```

Beispiele

```
touch datei  
[ \(-w datei -a -x datei\) ] || echo "w or x Bit or  
both missing on datei"
```

```
touch datei  
[ \(-w datei -o -x datei\) ] && echo "w or x Bit set on  
datei"
```

```
touch datei  
[ ! -d datei ] && echo "datei is not a directory "
```

Beispiel – Dateien umbenennen

von *.txt in *.dat in bestimmtes Verzeichnis

```
# Create sample files and remove if they exist
rm $1/a.dat $1/b.dat 2> /dev/null
touch $1/a.txt $1/b.txt

for F in `ls $1/*.txt`
do
    NEXT_FILE=`basename $1/$F`

    echo "Filename: $NEXT_FILE, Directory: $1 "

    NEW_FILE=`basename $1/$F | sed -E s/\.txt$/\.dat/`
    echo "New Filename:$1/${NEW_FILE}"

    mv $F $1/${NEW_FILE}
done
```